KU LEUVEN

Advanced Computer Architecture

Lab Report 2

Bert Pyck
Hugo Charmant

ELICT: ELECFS
27 februari 2025
Master in Engineering Technology
Academic year 2024-2025

1.1 Task description

The goal of this report is to implement maximum detection using three approaches:

- Using the CPU
- Using a GPU kernel employing atomic operations
- Using a GPU kernel with reduction

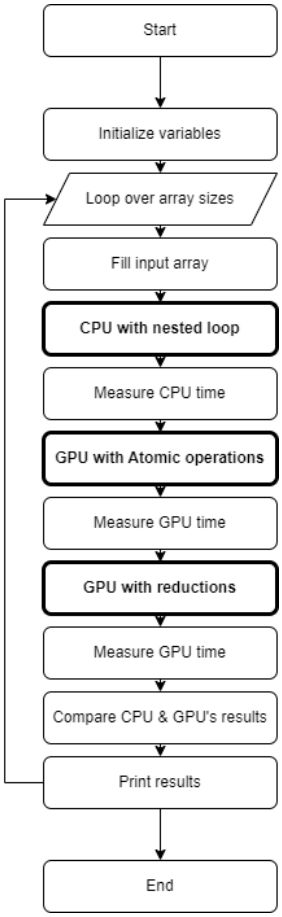
As in the first lab, we need to vary the 1D array size and analyze the time dependency. We will then present our findings, provide a comprehensive conclusion, and plot the results.

1.2 Introduction

We started the lab session by learning about race conditions. These are conditions that are necessary to handle multiple threads that want to access the same ressources. We learned how to deal with this problem by ressource locking and waiting as an alternative. Afterwards, we learned how to use atomic operation, to allow the threads to gain sequential access to the common ressource.

1.3 Code explanation

In this chapter, the explanation of the operation and implementation of the code is given. As explained in Figure 1.1, the idea is that we need to find the maximum value of an array. We will do this multiple times with the CPU and GPU. In order to achieve accurate measurements, we will perform the operations multiple times and compute an average. After that we will gradually change the size of the array and plot the results.



Figuur 1.1: Flowchart

1.3.1 Using the CPU

This function compares the elements of the array on the CPU *element-wise* with a max buffer value in a simple for-loop. It starts to initialize a maximal buffer value. Each element gets compared with the max buffer value. If an element is greater than this max value, the element replaces it. At the end of the for-loop, the max buffer value gets returned. A time also measures start and end time.

With an input array of 8, the output will be as follows:

```
input: [4,6,2,7,1,0,5,3]

for (int i = 0; i < 8; i++) {
    if (input[i] > maxValue){
        maxValue = input[i];
    }
}

maxValue = 7
```

1.3.2 Using a GPU kernel employing atomic operations

This function will run differently. Using CUDA to perform the operation on the GPU, it will find the maximum value in an array using atomic operations. It allocates memory on the GPU for the input array and a variable to store the maximum value. Following this, the data is copied and the kernel calculates the thread index. A CUDA kernel is launched with multiple threads, each comparing an element from the array and updating the maximum value using the atomicMax function. With the use of the atomic max function, the threads will get sequential acces to the maximum value. This ensures that concurrent threads update the maximum safely without a race condition. After execution, the computed maximum value is copied back and displayed. It is crucial to finish this sequency with garbage collection at the end.

From the "CUDA C++ Programming Guide", pages 189, we learn that the atomicMax() function takes the same approach as our CPU function. It combines the data fetching, comparison and writing in one operation.

With an input array [4,6,2,7,1,0,5,3], the output will be as follows:

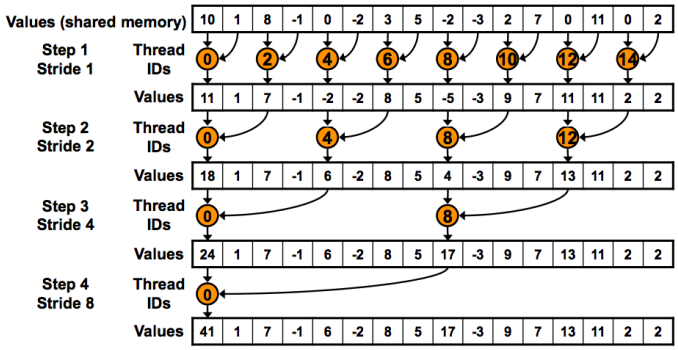
Thread (t_id)	**d_input[t_id]**	**atomicMax(d_maxVal, d_input[t_id])**
0	4	max(0, 4) $\rightarrow$ 4
1	6	max(4, 6) $\rightarrow$ 6
2	2	max(6, 2) $\rightarrow$ 6
3	7	max(6, 7) $\rightarrow$ 7
4	1	max(7, 1) $\rightarrow$ 7
5	0	max(7, 0) $\rightarrow$ 7
6	5	max(7, 5) $\rightarrow$ 7
7	3	max(7, 3) $\rightarrow$ 7

Tabel 1.1: GPU AtomicMax

1.3.3 Using a GPU kernel with reduction

This computation tackles the race condition problem in a different way. By reducing the parallel problem to a *semi-parallel* one, it eliminates the possibility of accessing the same memory at the same time. In order to implement this function correctly, we tackle the problem step by step, using a sync thread at each iteration to make sure there is no race condition taking place.

We expect this implementation to be the fastest, since it utilizes parallellism best. Especially with arrays of length 2^N.

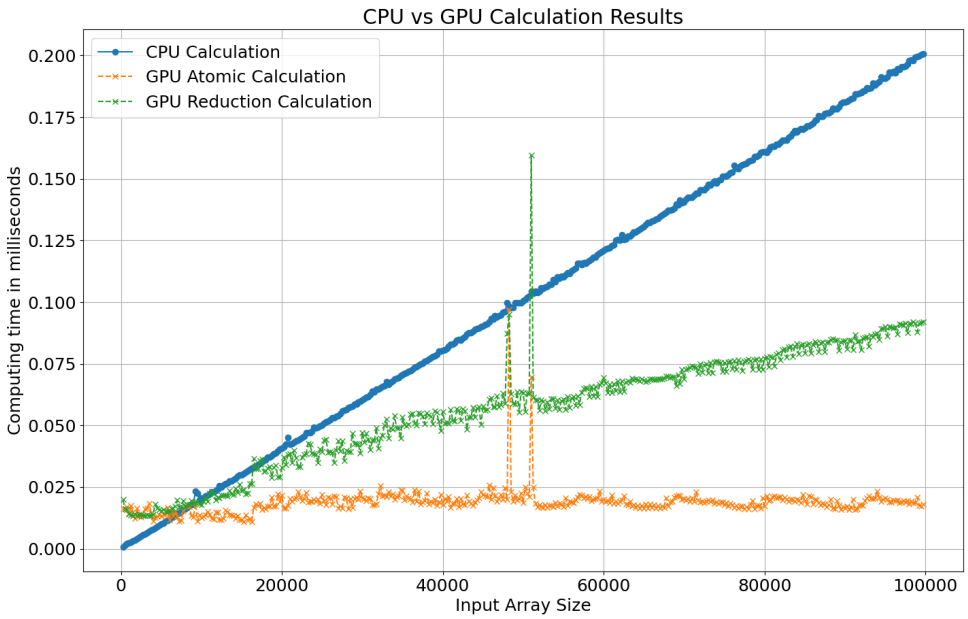


Figuur 1.2: Example of reduction for addition (source: Mateuszbuda GitHub)

1.4 Results and conclusion

1.4.1 Are the results correct?

From the results in figure 1.3 we can see that the time it takes for the CPU to compute increase linearly with the size of the array. This means that the time complexity of our operation is of N, which is correct.
For the GPU calculation with atomic operation, we see that the computing time is almost independent of the array size. For the reduction, the complexity seems to follow the expected trend, altough it is slower than the atomic operation.



Figuur 1.3: Results of our calculations

1.4.2 What happens if input size is larger than 2048?

A normal maximal block size is 1024. In a situation with more than 2048 elements in the array we would simply perform the maximum calculation, moving along the array iteration per iteration.

1.4.3 What happens when using less threads?

Our proposed approach in this situation would be to perform the same calculations, but on parts of the array. This would mean that we reperform the calculation, but simply moving along the array.

1.4.4 How would you tackle a 2D array?

Our proposed approach for this situation would be to first perform the computations row per row, followed by a final column calculation containing the maxima of each row.

An alternative method, utilizing multi-dimensional thread blocks, would be to create a 2D thread block and to iterate over the whole 2D array. Using a grid consisting of multiple of these 2D-blocks, we could achieve even greater processing speeds. An important limitation of this method is that the dimensions of our 2D-array need to be a multiple of the dimensions of a 2D- block. Our proposed solution to achieve this is to zero-pad the array (doesn't change the outcome for a maximum search).

While looking in the documentation, we found that the cudaMallocPitch() function automatically appropriately pads the array to fit with our thread and block dimensions.

2 CODE

```cpp
#include <iostream>
#include <cuda_runtime.h>
#include <chrono>

#define BLOCK_SIZE 1024
#define AVERAGE 1000

using namespace std;

unsigned int SIZE = 100;

double resultCPU = 0;
double resultGPUatomic = 0;
double resultGPUreduction = 0;

std::pair<long int, double> cpu_max(unsigned int input[]) {
    long int maxValue = 0;
    const auto start = std::chrono::steady_clock::now();
    for (int i = 0; i < SIZE; i++) {
        if (input[i] > maxValue) {
            maxValue = input[i];
        }
    }
    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed_seconds = end - start;
    double timeCount = elapsed_seconds.count();

    return {maxValue, timeCount};
}

__global__ void gpu_atomic_max(unsigned int *d_input, unsigned int *d_maxVal, int
     size) {
    int t_id = threadIdx.x + blockIdx.x * blockDim.x;

    if (t_id < size) {
        atomicMax(d_maxVal, d_input[t_id]);
    }
}

__global__ void gpu_reduction_max(unsigned int *d_input, unsigned int *d_output,
    int size) {
    extern __shared__ unsigned int s_data[];

    int tid = threadIdx.x;
    // Compute starting index for this block. Each thread will load two elements.
```

```
    int i = blockIdx.x * (blockDim.x * 2) + tid;

    // Load two elements per thread into shared memory (if within bounds)
    unsigned int myMax = 0;
    if (i < size)
        myMax = d_input[i];
    if (i + blockDim.x < size)
        myMax = max(myMax, d_input[i + blockDim.x]);
    s_data[tid] = myMax;
    __syncthreads();

    // Perform tree-based reduction in shared memory in logarithmic steps.
    // Start with half the block size and reduce by factors of 2.
    for (unsigned int s = blockDim.x / 2; s > 32; s >>= 1) {
        if (tid < s) {
            s_data[tid] = max(s_data[tid], s_data[tid + s]);
        }
        __syncthreads();
    }

    // Write the block's maximum to the output.
    if (tid == 0)
        d_output[blockIdx.x] = s_data[0];
}

int main(int argc, char **argv) {
    for(int j = 1; j < 100000; j=j+250){
        // std::cout << "SIZE;" << "CPU;" << "GPU" << std::endl;
        SIZE = j;
        unsigned int input[SIZE];

        unsigned int seed = static_cast<unsigned int>(
            std::chrono::system_clock::now().time_since_epoch().count()
        );

        srand(seed); // Seed the random number generator
        for (int i = 0; i < SIZE ; i++){
            input[i] = rand();
            // std::cout << input[i] << std::endl;
        }
        // std::cout << "This was the array" << std::endl;


        for(int i = 0; i < AVERAGE; i++){
            auto result1 = cpu_max(input);
            resultCPU = resultCPU + result1.second;
        }
        resultCPU = resultCPU / AVERAGE;

        // std::cout << result1.first << " is the max value from the CPU" << std::
            endl;

        // GPU atomic operation
        unsigned int *d_input, *d_maxVal;
        unsigned int h_maxVal = 0;
        int numBlocks = 1;
```

```
99
100         for(int i = 0; i < AVERAGE; i++){
101             cudaEvent_t start_cuda, stop_cuda;
102             cudaEventCreate(&start_cuda);
103             cudaEventCreate(&stop_cuda);
104
105             cudaMalloc(&d_input, SIZE * sizeof(unsigned int));
106             cudaMalloc(&d_maxVal, sizeof(unsigned int));
107
108             // Copy data from host to device
109             cudaMemcpy(d_maxVal, &h_maxVal, sizeof(unsigned int),
                    cudaMemcpyHostToDevice);
110             cudaMemcpy(d_input, input, SIZE * sizeof(unsigned int),
                    cudaMemcpyHostToDevice);
111
112             cudaEventRecord(start_cuda);
113
114             // Launch kernel (calculate required blocks)
115             // int numBlocks = (SIZE + BLOCK_SIZE - 1) / BLOCK_SIZE;
116             gpu_atomic_max<<<numBlocks, BLOCK_SIZE>>>(d_input, d_maxVal, SIZE);
117
118             cudaDeviceSynchronize();
119
120             cudaEventRecord(stop_cuda);
121             cudaEventSynchronize(stop_cuda);
122
123             cudaMemcpy(&h_maxVal, d_maxVal, sizeof(unsigned int),
                    cudaMemcpyDeviceToHost);
124             // std::cout << h_maxVal << " is the max value from the GPU with
                    atomicMax" << std::endl;
125
126             // Free device memory
127             cudaFree(d_input);
128             cudaFree(d_maxVal);
129
130             float ms;
131             cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
132             resultGPUatomic += ms/1000;
133         }
134
135         resultGPUatomic = resultGPUatomic / AVERAGE;
136
137         for (int i = 0; i < AVERAGE; i++) {
138             cudaEvent_t start_cuda, stop_cuda;
139             cudaEventCreate(&start_cuda);
140             cudaEventCreate(&stop_cuda);
141
142             unsigned int *d_input, *d_output;
143             cudaMalloc(&d_input, SIZE * sizeof(unsigned int));
144             cudaMemcpy(d_input, input, SIZE * sizeof(unsigned int),
                    cudaMemcpyHostToDevice);
145
146             int numBlocks = (SIZE + BLOCK_SIZE - 1) / BLOCK_SIZE;
147             cudaMalloc(&d_output, numBlocks * sizeof(unsigned int));
148
149             cudaEventRecord(start_cuda);
```

```
150
151             gpu_reduction_max<<<numBlocks, BLOCK_SIZE, BLOCK_SIZE * sizeof(
                    unsigned int)>>>(d_input, d_output, SIZE);
152             cudaDeviceSynchronize();
153
154             cudaEventRecord(stop_cuda);
155             cudaEventSynchronize(stop_cuda);
156
157             float ms;
158             cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
159             resultGPUreduction += ms / 1000;
160
161             unsigned int h_output[numBlocks];
162             cudaMemcpy(h_output, d_output, numBlocks * sizeof(unsigned int),
                    cudaMemcpyDeviceToHost);
163
164             cudaFree(d_input);
165             cudaFree(d_output);
166         }
167
168     resultGPUreduction /= AVERAGE;
169     std::cout << SIZE << "," << resultCPU << "," << resultGPUatomic << "," <<
            resultGPUreduction << std::endl;
170     }
171     return 0;
172 }
```

Listing 2.1: The maximum Cuda code